

Microservices Interview

Questions

Q1. How would you design Netflix-like streaming architecture?

Ans. 1. Functional Requirements

Core Features

- User Signup/Login
- Browse/Search content
- Video playback
- Resume watching
- Recommendations
- Multiple device support
- Adaptive bitrate streaming

Non-Functional Requirements

- High availability (99.99%)
- Low latency streaming
- Massive scalability (millions of users)
- Fault tolerance
- Global delivery

- Secure content delivery (DRM)



1 Client Layer (Web / Mobile / TV)

- Web browser
- Mobile app
- Smart TV app

What happens here:

- User logs in
- Browses content
- Clicks Play
- Streams video

All communication happens over HTTPS APIs.

👉 These clients never directly talk to databases or internal services.

They call:

Client → API Gateway

2 API Gateway (Entry Point)

This is the single entry point to the system.

Responsibilities:

- Authentication validation (JWT)
- Rate limiting
- Routing request to correct service
- Logging & monitoring

Example:

GET /movies → routed to Content Service

POST /login → routed to Auth Service

👉 This keeps internal services hidden and secure.

Core Microservices Layer

Now we enter the business logic layer.

The diagram shows 3 major services:

A. Auth Service

Purpose:

- Login
- Signup
- Token generation
- Session validation

Database:

- User Database (Relational DB)

Flow:

Login Request → Auth Service → User DB → JWT
Generated

B. Content Service

Purpose:

- Fetch movie metadata
- Genre list
- Movie details
- Thumbnails

Database:

- Metadata Database (NoSQL)

Important:

This service does NOT stream video.

It only returns metadata like:

- Title
- Description
- Duration
- Thumbnail URL



C. Recommendation Service

Purpose:

- Personalized suggestions
- Trending movies
- "Because you watched..."

Data Source:

- Watch history
- User behavior
- ML models

It connects to:

- Recommendations Engine (ML backend)



Data Layer (Databases)

Each microservice has its own database.

This is VERY IMPORTANT in microservices.

Auth Service → User DB

Content Service → Metadata DB

Recommendation Service →

ML/Recommendations DB

Why separate databases?

- Independent scaling
- Loose coupling
- Service autonomy

5 Video Storage & Transcoding Layer

This is where the real video files live.

Video Storage

Stored in:

- Object storage (like S3)

Stores:

- Raw uploaded videos
- Processed versions

Transcoding Pipeline

When a video is uploaded:

Upload → Storage → Transcoding → Multiple
Formats

Video gets converted into:

- 240p
- 480p
- 720p

- 1080p
- 4K

Why?

→ Adaptive Bitrate Streaming

If internet is slow → lower quality

If internet is fast → higher quality



Global CDN (Most Important Part)

After transcoding, videos are pushed to:

Global CDN

CDN means:

Content Delivery Network

Purpose:

- Store videos closer to users
- Reduce latency
- Reduce load on backend servers

Flow during playback:

User clicks Play



Streaming Service generates signed URL



Client fetches video from CDN directly

⚠ IMPORTANT INTERVIEW POINT:

Backend APIs are NOT in the streaming path.

Streaming goes directly from CDN → User.



End Users

Users receive:

- Video chunks from CDN
- Adaptive streaming
- Smooth playback

Final Interview-Ready Answer (Structured)

I would design Netflix using a microservices architecture with API Gateway as entry point, separate services for auth, content, recommendations, search, and streaming. Videos would be uploaded to object storage, processed asynchronously via Kafka-based transcoding pipeline, and distributed globally through CDN. All services would be stateless and horizontally scalable, with Redis caching and NoSQL databases for high throughput. Streaming traffic would bypass backend APIs and directly hit CDN for low latency. The system would use multi-region deployment, circuit breakers, and chaos engineering for high availability.



What is a CDN?

A **Content Delivery Network (CDN)** is a globally distributed network of edge servers that cache and deliver content to users from the nearest geographic location.

Example:

- Open Connect (built by Netflix)
 - Cloudflare
 - Akamai Technologies
-

Role of CDN in Netflix-like Architecture

1 Reduce Latency (Faster Streaming)

Without CDN:

User (India) → US Data Center → Video → Back to India

With CDN:

User (India) → Local Edge Server (Mumbai)

👉 Video starts instantly.

👉 No buffering.

👉 Low startup delay.

2 Massive Traffic Handling (Scalability)

Imagine:

- 10 million users watching the same movie.

Without CDN:

- Backend storage & servers crash.

With CDN:

- Video is cached at thousands of edge locations.
- Each region handles its own traffic.

Backend is protected.

Q2. What happens if one service is down?

Ans. In a microservices architecture, if one service goes down, the system should degrade gracefully rather than fail completely. We use resilience patterns such as circuit breakers, timeouts, retries, bulkheads, caching, and load balancing to prevent cascading failures. Non-critical services can return fallback responses, while critical services rely on redundancy and health checks to recover automatically. The goal is fault isolation and high availability.

Q3. How do you secure microservices?

- Ans. All external requests go through an **API Gateway** for authentication and rate limiting.
- Use **JWT or OAuth2** for authentication and **RBAC** for authorization.
- Secure service-to-service communication with **HTTPS ,mTLS or A3**.
- Encrypt data **in transit and at rest**.
- Store secrets in a **secure vault**, not in code.
- Enable logging, monitoring, and rate limiting for attack detection.

The key principle is **Zero Trust — verify every request**.

Q4. What tools have you used for monitoring?

- Ans. **Prometheus** for metrics collection
- **Grafana** for dashboards and visualization
- **ELK Stack** (Elasticsearch, Logstash, Kibana) for centralized logging
- **Splunk** for log analysis (in enterprise setups)
- **Spring Boot Actuator** for application health and metrics

- **New Relic / Datadog** for APM (Application Performance Monitoring)
- **Kubernetes** monitoring tools for pod health and resource usage

These tools help monitor metrics, logs, traces, alerts, and system health in a microservices environment.

Q5. How do you handle distributed transactions?

Ans. ? Why Distributed Transactions Are Difficult?

In microservices:

- Each service has its own database.
- A single business operation may span multiple services.
- No global database transaction.

Example:

Order Service → Payment Service → Inventory Service

If payment succeeds but inventory fails, system becomes inconsistent.



Two-Phase Commit (2PC)

2PC ensures atomicity across multiple services/databases.



Phase 1: Prepare Phase (Voting)

Coordinator asks all participants:

“Can you commit?”

Participants:

- Lock resources
 - Prepare transaction
 - Respond YES / NO
-



Phase 2: Commit Phase

If all say YES:

- Coordinator sends COMMIT

If any says NO:

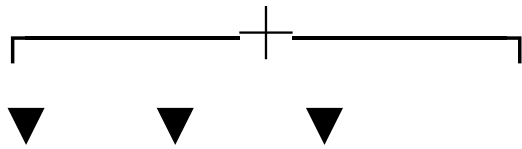
- Coordinator sends ROLLBACK
-



2PC Diagram

Coordinator

|



Service A Service B Service C

Phase 1:

Coordinator → PREPARE

Participants → YES / NO

Phase 2:

If all YES → COMMIT

Else → ROLLBACK

✗ Why 2PC Is Avoided in Microservices?

Major problems:

1 Blocking Protocol

If coordinator crashes:

- Participants stay locked.
- System blocks.

2 Performance Bottleneck

- Locks held for long time.

- Reduces throughput.

Not Cloud-Friendly

- Doesn't scale well in distributed environments.
- Poor fit for microservices.

Single Point of Failure

Coordinator failure impacts all.

👉 That's why modern microservices avoid 2PC.

2 Three-Phase Commit (3PC)

3PC tries to improve 2PC by reducing blocking.

Phases:

1 CanCommit?

2 PreCommit

3 DoCommit

3PC Diagram

Coordinator

|

CanCommit?

|

Participants vote

|

PreCommit

|

Participants acknowledge

|

DoCommit

Why 3PC is Better?

- Reduces blocking
- Adds timeout handling

✗ Why 3PC Is Rarely Used?

- Complex
- Still not perfect in network partitions
- Expensive
- Not widely supported

In cloud-native systems, still not preferred.



3

Saga Pattern (Most Preferred in Microservices)

Instead of locking everything, we use:

Sequence of local transactions + compensating transactions

Each service:

- Commits locally
 - If failure happens → execute compensating action
-

Saga Example: Order System

Steps:

- 1** Create Order
- 2** Deduct Inventory
- 3** Process Payment

If payment fails:

- Compensate inventory
 - Cancel order
-

Saga Types

A. Choreography-Based Saga

No central coordinator.

Services communicate via events.

Order Service → Event

Inventory Service → Event

Payment Service → Event

Uses event broker like

Apache Kafka

Diagram

Order → Event → Inventory → Event → Payment



Compensation Event if failure

✓ Loose coupling

✓ Highly scalable

✗ Hard to debug

B. Orchestration-Based Saga

Central Saga Orchestrator controls flow.

Saga Orchestrator



Order → Inventory → Payment

If failure:

- Orchestrator triggers compensations.

- ✓ Easier to manage
 - ✓ Clear flow
 - ✗ Slight central control
-

Saga Architecture Diagram

Client



Order Service (Local Commit)



Event / Orchestrator



Inventory Service



Payment Service



Compensation if needed

⚖️ 2PC vs 3PC vs Saga

Feature	2PC	3PC	Saga
Blocking	Yes	Reduced	No

Feature	2PC	3PC	Saga
Scalability	Poor	Medium	High
Cloud Friendly	No	Rare	Yes
Performance	Low	Medium	High
Microservices Fit	✗	✗	✓

When To Use What?

Use 2PC When:

- Monolithic system
 - Strong ACID required
 - Limited scale
 - Banking core systems
-

Use 3PC When:

- Rare case
- Distributed DB systems
- Need timeout-based improvement

(Not common in microservices)

✅ Use Saga When:

- Microservices architecture
- High scalability required
- Event-driven systems
- Cloud-native applications

Most modern systems use Saga.

🎤 Interview-Ready Answer

You can say:

“In microservices, distributed transactions are handled using patterns like Saga instead of traditional 2PC. 2PC ensures atomicity but is blocking, reduces scalability, and doesn’t work well in cloud-native systems. 3PC improves on 2PC by adding a pre-commit phase, but it is complex and still not ideal for network partitions. Therefore, modern microservices prefer the Saga pattern, where each service performs a local transaction and compensating actions are executed in case of failure. Saga can be implemented using choreography with events or orchestration with a central coordinator.”

Sample Project: E-Commerce Order System

We have 3 microservices:

1. Order Service → Creates order
2. Inventory Service → Reserves stock
3. Payment Service → Charges customer

Business flow:

Create Order → Reserve Inventory → Process Payment

If payment fails → rollback everything.

Now let's implement this using:

- 2PC
 - 3PC
 - Saga (Choreography)
 - Saga (Orchestration)
-

**1**

Two-Phase Commit (2PC) – Real Implementation

When used?

Mostly in distributed databases (not typical microservices REST).

Setup

All services use:

- Same transaction coordinator
- XA-compliant databases
- JTA (Java Transaction API)

Example stack:

- Spring Boot
- Atomikos / Narayana (Transaction Manager)
- XA-enabled MySQL/Postgres



Flow

Phase 1: Prepare

Coordinator:

START GLOBAL TRANSACTION

Calls:

Order DB → prepare()

Inventory DB → prepare()

Payment DB → prepare()

Each:

- Locks row
- Writes to transaction log
- Says YES / NO

Phase 2: Commit

If all YES:

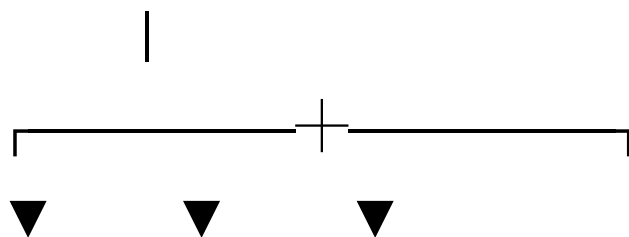
COMMIT

Else:

ROLLBACK

2PC Diagram

Transaction Manager



Order DB Inventory DB Payment DB

Prepare → Vote → Commit/Rollback

❌ Problem in Real Microservices

If Payment service crashes after prepare:

- All DB rows remain locked.
- System blocks.

That's why 2PC is rarely used in microservices.

🧱 2 Three-Phase Commit (3PC)

Adds extra step to reduce blocking.

Phases:

1. CanCommit?
 2. PreCommit
 3. DoCommit
-

Flow Example

Coordinator:

CanCommit?

All say YES →

Coordinator:

PreCommit

Participants acknowledge →

Coordinator:

DoCommit

Why Rare?

- Complex
 - Still fails under network partition
 - Hard to implement manually
 - Not supported widely in microservices frameworks
-

  Saga Pattern – Choreography (Real Example)

This is most common in modern systems.

Uses:

 Apache Kafka

 Flow

Step 1 – Order Created

Order Service:

Save order (status=PENDING)

Publish Event: OrderCreated

Step 2 – Inventory Listens

Inventory Service:

On OrderCreated:

Reserve stock

If success → Publish InventoryReserved

If fail → Publish InventoryFailed

Step 3 – Payment Listens

Payment Service:

On InventoryReserved:

Charge payment

If success → Publish PaymentCompleted

If fail → Publish PaymentFailed

Compensation Example

If PaymentFailed:

Order Service listens:

Update order status = CANCELLED

Inventory Service listens:

Release stock

Saga Choreography Diagram

Order → (OrderCreated Event)



Inventory → (InventoryReserved Event)



Payment → (PaymentCompleted)

If failure:

PaymentFailed → Order Cancel + Stock Release

✓ No global lock

✓ Fully async

✓ Scalable

Code Example (Spring Boot + Kafka)

Order Service publishes:

```
kafkaTemplate.send("order-created", orderEvent);
```

Inventory consumes:

```
@KafkaListener(topics = "order-created")  
public void reserveStock(OrderEvent event) {  
    // reserve logic  
}
```



Saga Pattern – Orchestration (Real Example)

Instead of events triggering automatically,
we use a Saga Orchestrator Service.



Flow

Saga Orchestrator:

1. Call Order Service
2. Call Inventory Service
3. Call Payment Service

If payment fails:

Call Inventory compensation

Call Order compensation



Orchestration Diagram

Saga Orchestrator

